Group 7 / C303-301G

CHATBOT
HUMAN TO HUMAN

Professor: Amor Ellares

Table Of Content

About Our Project
01
This is a human-to-human chatroom application. Our software system operates with a graphical user interface (GUI) and allows multiple users to join and communicate through a centralized server.

Our great Team
02
Our group makes it more engaging with a user-friendly GUI and features like emojis and much more.

About Our Technology
03
The application uses Python for simplicity, with Tkinter for the GUI, sockets for real-time communication, and Pillow for resizing and displaying logos.

Function Technology
04
The application offers real-time messaging, emoji support, a server-client architecture for multi-user interaction, and a user-friendly interface for seamless communication.

Project Stage
05
The project has moved from development, with core features functional, to testing and refinement, focusing on optimization, bug fixes, and UI improvements.

Project Goals
06
Create an intuitive and interactive chatroom interface with real-time communication, emoji support for engaging messaging, and scalability to handle multiple users simultaneously.

Our Great Team

The project is a human-to-human chatroom application built with Python, using Tkinter for the GUI and sockets for client-server communication, featuring a user-friendly interface, emoji support, typing indicators, and real-time message handling with threading.



Miguel

DEVELOPER

Miguel handled core functionalities like socket connections, message exchange, and integrating the emoji picker and send button.



Anthon

DEVELOPER

Anthon focused on the GUI design, ensuring a visually appealing and easy-to-navigate chatroom with smooth transitions.

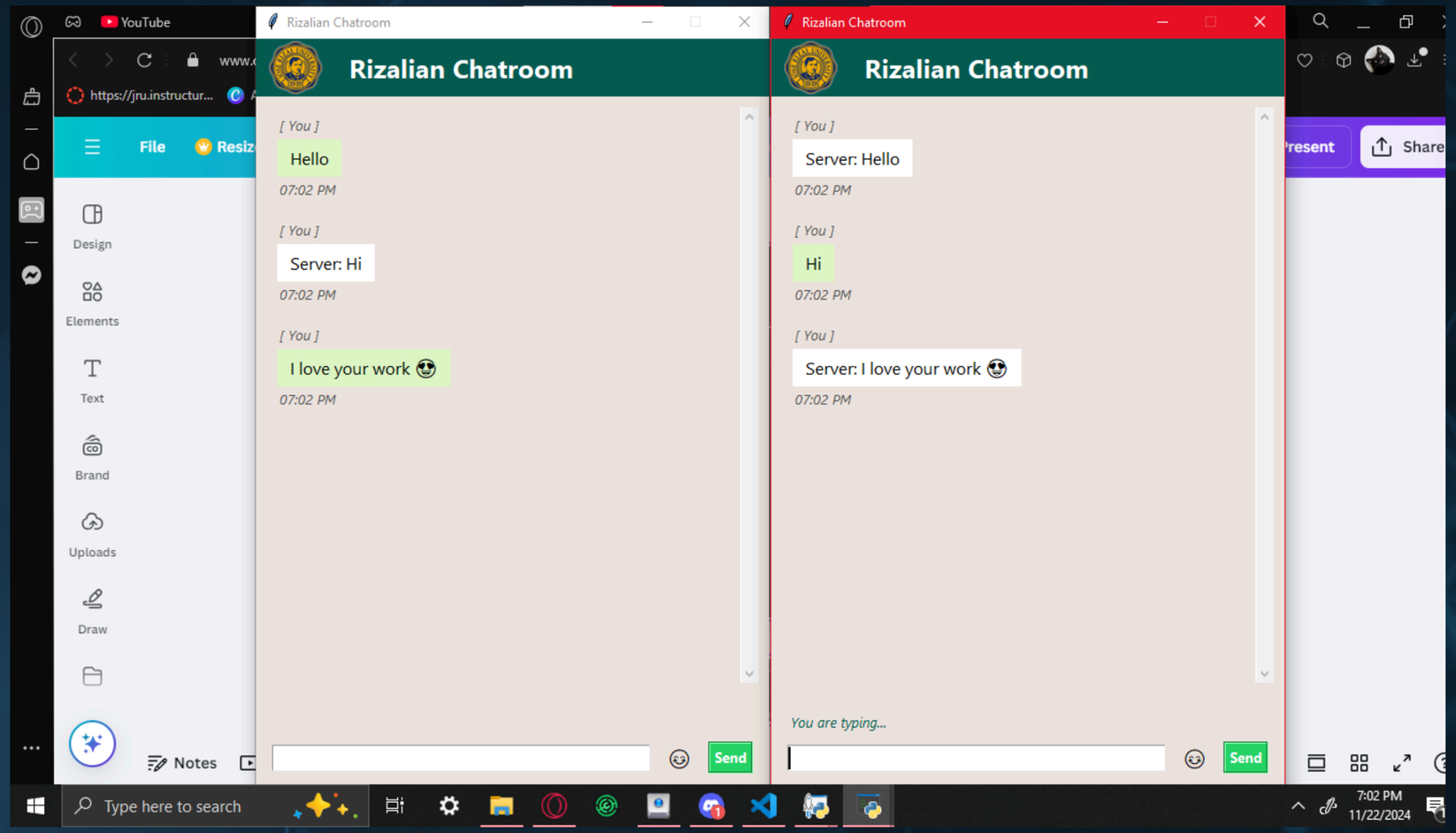


Harold

DEVELOPER

Harold worked on message formatting, including message bubbles, alignment, timestamps, and handling multiple users.

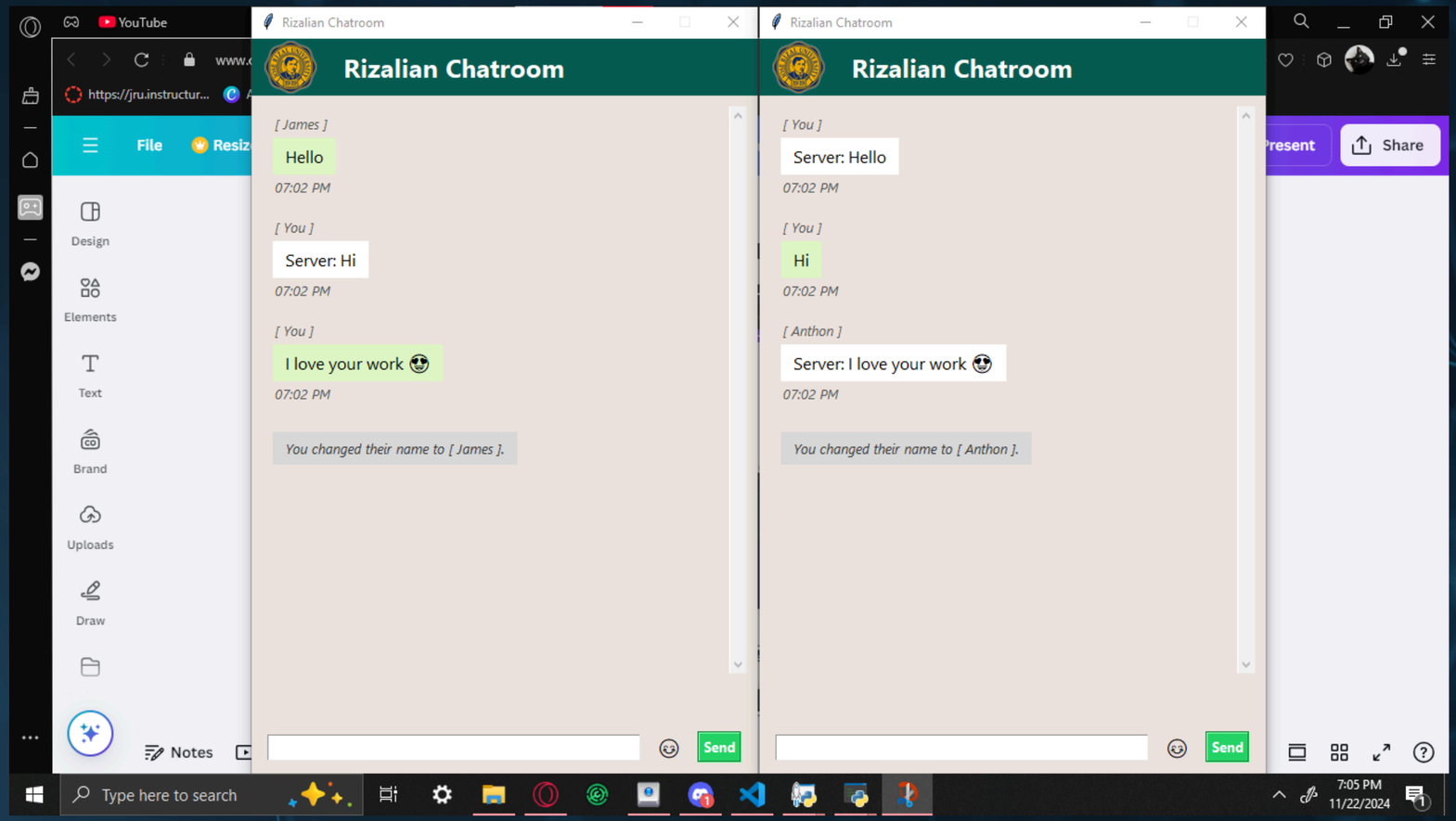
User Interface



Descriptions

The image shows the user interface of the "Rizalian Chatroom," where a user and server interact in separate chat windows. The chat is visually separated with message timestamps, and the interface features a user-friendly layout with message composition and a typing indicator.

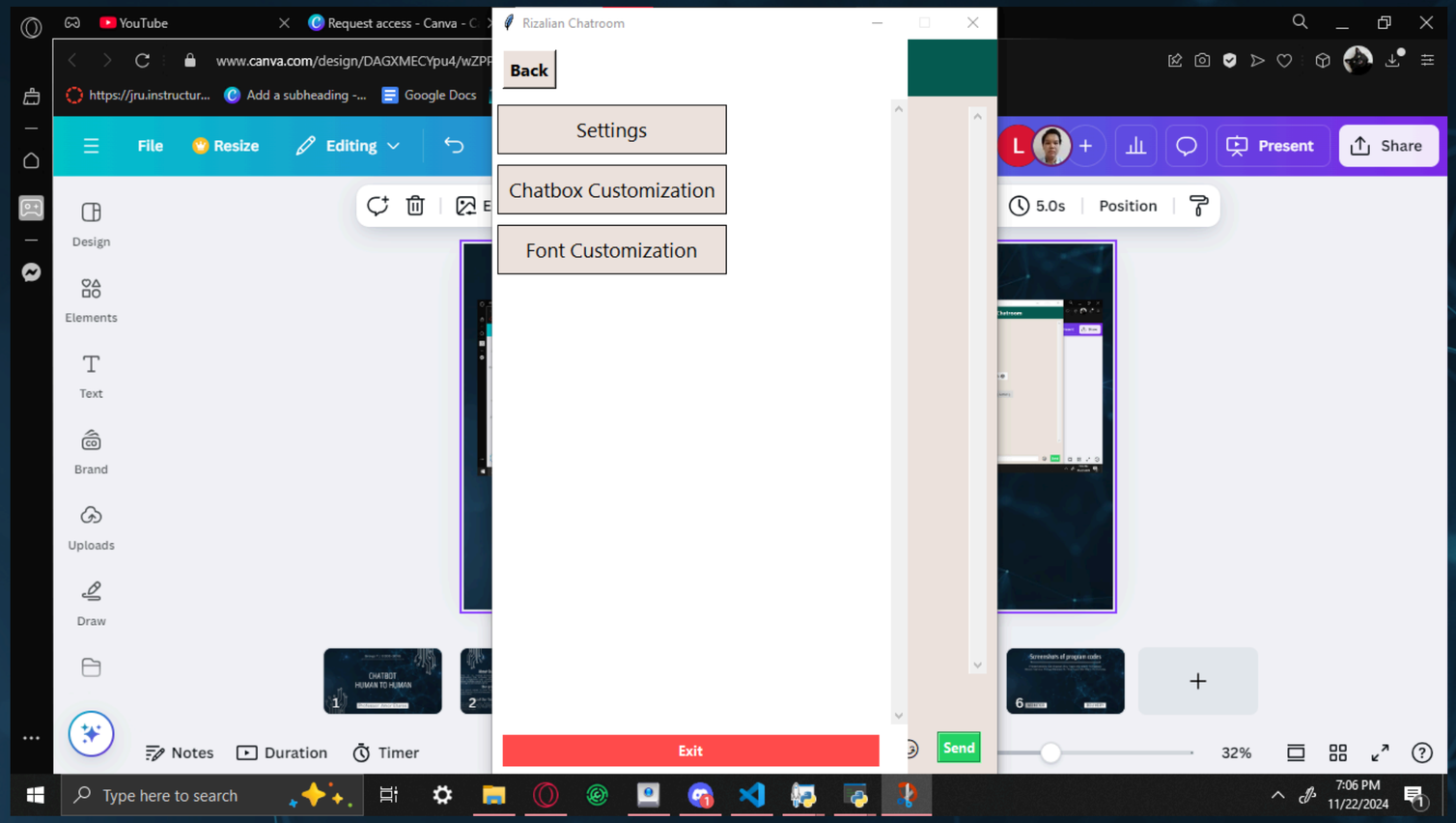
User Interface



Descriptions

The image displays an updated version of the "Rizalian Chatroom" interface, where users can now change their display names. The change in usernames is highlighted in the chat window, showing real-time updates for the user and server messages.

User Interface



Descriptions

This is the settings menu of the Ricalan Chatroom application, featuring a Back button, options for Settings, Chatbox Customization, and Font Customization, and a red Exit button at the bottom. The interface is simple and user-friendly, allowing easy navigation and customization.

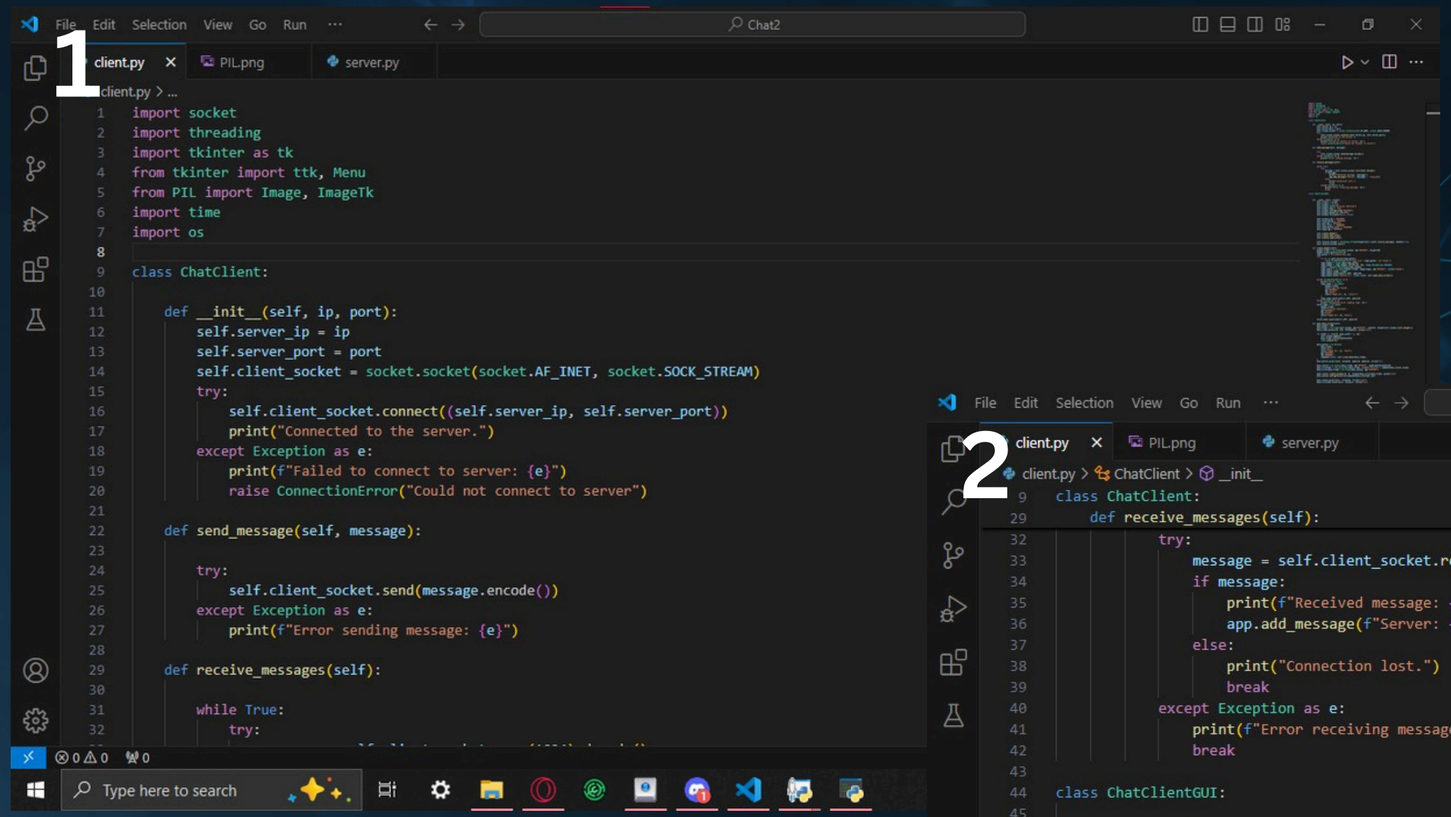


CLIENT CODES

The client-side application allows users to connect to the chat server and send messages to other participants in real-time. It displays received messages from the server or other clients and enables users to type and send messages through a simple chat interface.

Screenshots of program codes for Clients

1



```python
import socket
import threading
import tkinter as tk
from tkinter import ttk, Menu
from PIL import Image, ImageTk
import time
import os

class ChatClient:

    def __init__(self, ip, port):
        self.server_ip = ip
        self.server_port = port
        self.client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        try:
            self.client_socket.connect((self.server_ip, self.server_port))
            print("Connected to the server.")
        except Exception as e:
            print(f"Failed to connect to server: {e}")
            raise ConnectionError("Could not connect to server")

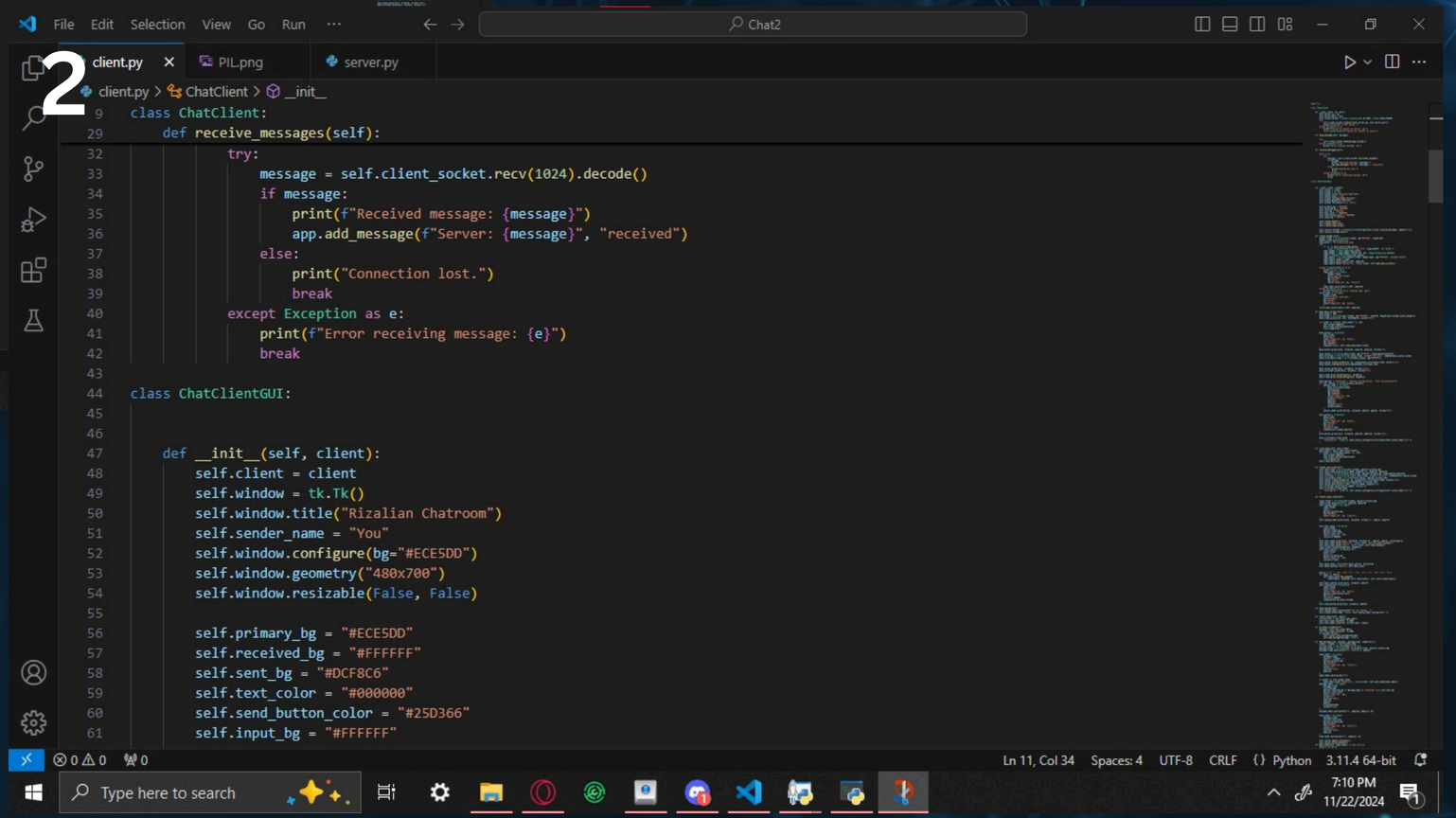
    def send_message(self, message):

        try:
            self.client_socket.send(message.encode())
        except Exception as e:
            print(f"Error sending message: {e}")

    def receive_messages(self):

        while True:
            try:
```

2



```python
class ChatClient:
    def receive_messages(self):
            try:
                message = self.client_socket.recv(1024).decode()
                if message:
                    print(f"Received message: {message}")
                    app.add_message(f"Server: {message}", "received")
                else:
                    print("Connection lost.")
                    break
            except Exception as e:
                print(f"Error receiving message: {e}")
                break

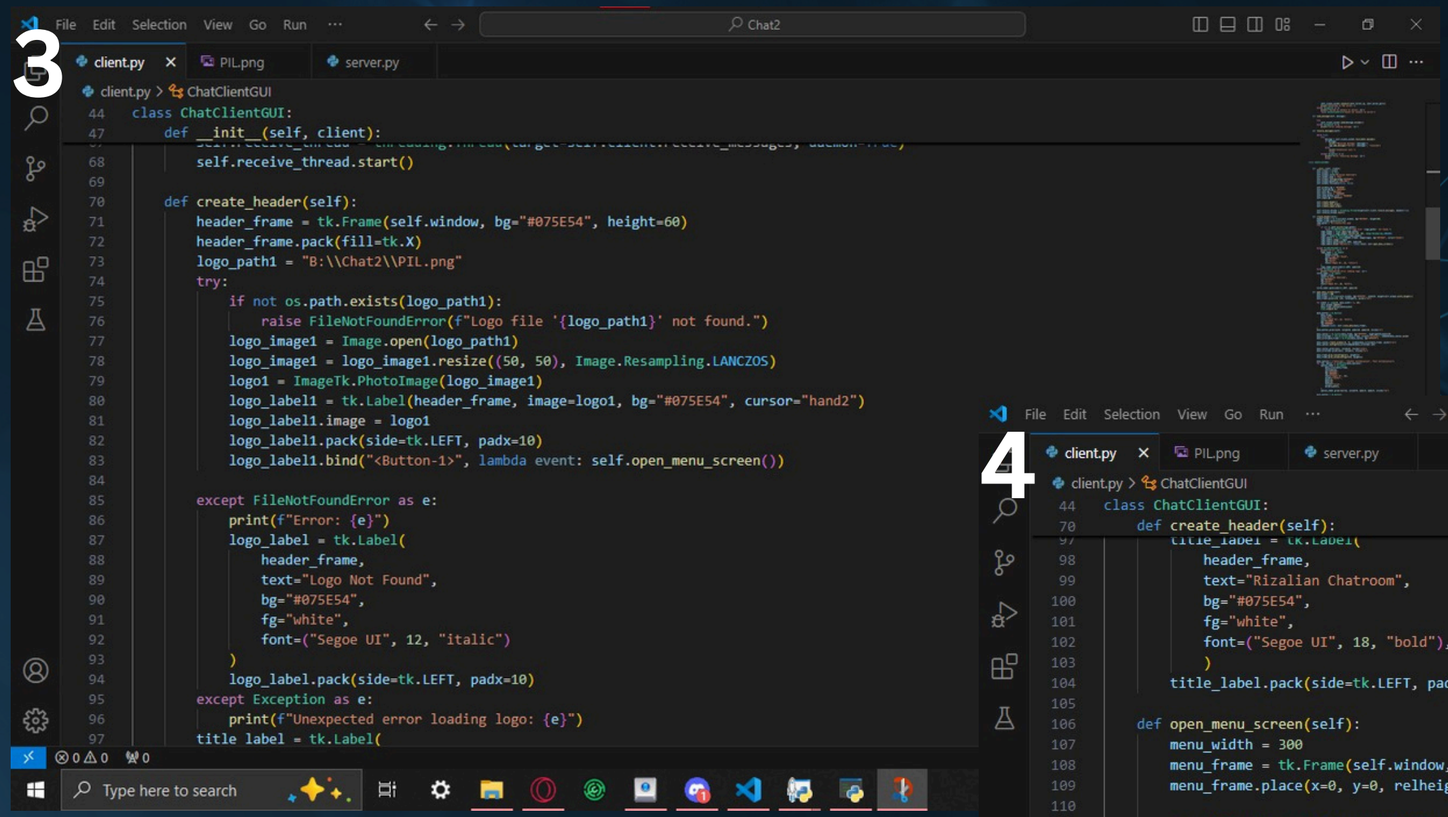
class ChatClientGUI:


    def __init__(self, client):
        self.client = client
        self.window = tk.Tk()
        self.window.title("Rizalian Chatroom")
        self.sender_name = "You"
        self.window.configure(bg="#ECE5DD")
        self.window.geometry("480x700")
        self.window.resizable(False, False)

        self.primary_bg = "#ECE5DD"
        self.received_bg = "#FFFFFF"
        self.sent_bg = "#DCF8C6"
        self.text_color = "#000000"
        self.send_button_color = "#25D366"
        self.input_bg = "#FFFFFF"
```

Screenshots of program codes for Clients

3

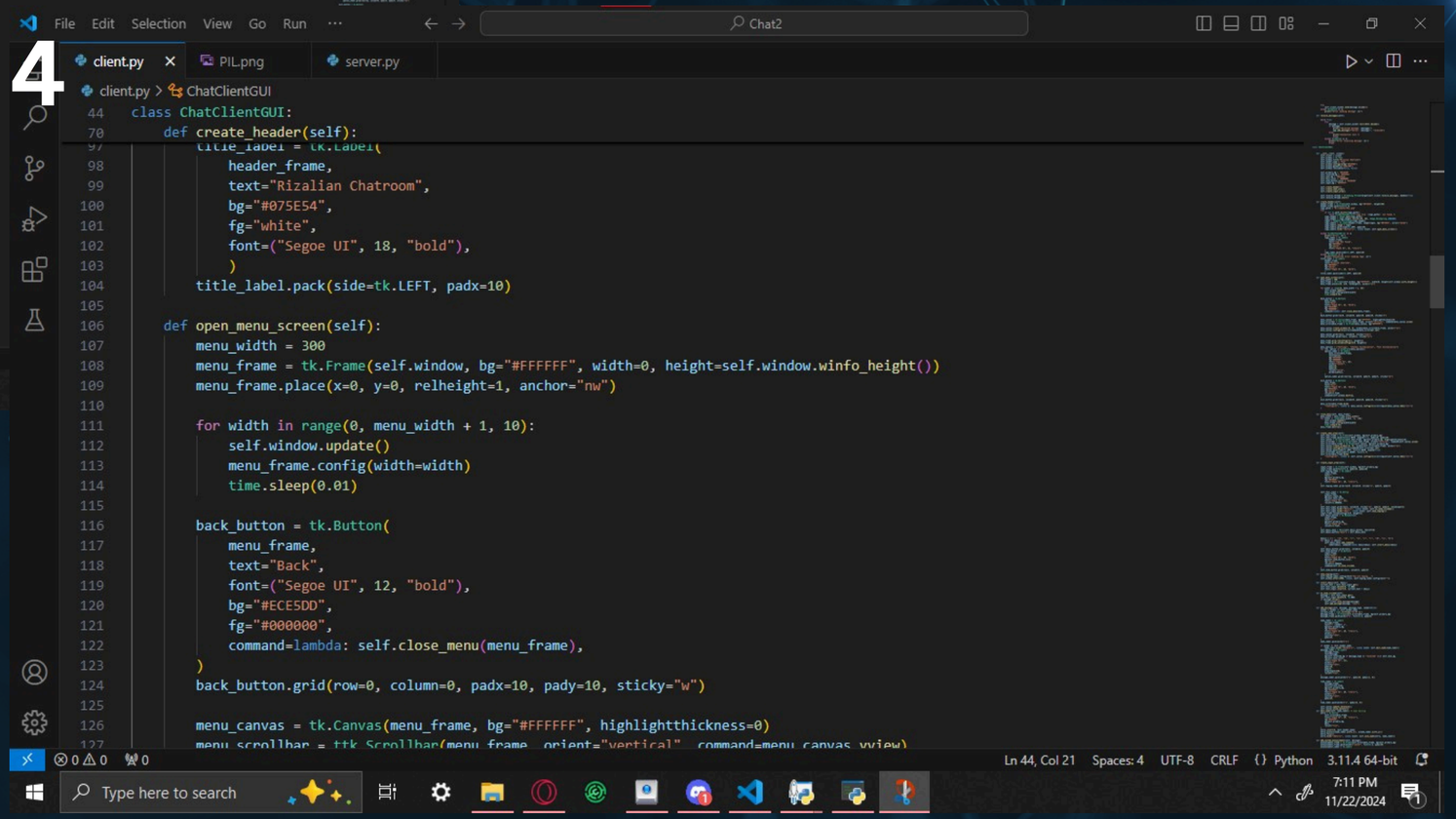


```
class ChatClientGUI:
    def __init__(self, client):
        self.receive_thread.start()

    def create_header(self):
        header_frame = tk.Frame(self.window, bg="#075E54", height=60)
        header_frame.pack(fill=tk.X)
        logo_path1 = "B:\\Chat2\\PIL.png"
        try:
            if not os.path.exists(logo_path1):
                raise FileNotFoundError(f"Logo file '{logo_path1}' not found.")
            logo_image1 = Image.open(logo_path1)
            logo_image1 = logo_image1.resize((50, 50), Image.Resampling.LANCZOS)
            logo1 = ImageTk.PhotoImage(logo_image1)
            logo_label1 = tk.Label(header_frame, image=logo1, bg="#075E54", cursor="hand2")
            logo_label1.image = logo1
            logo_label1.pack(side=tk.LEFT, padx=10)
            logo_label1.bind("<Button-1>", lambda event: self.open_menu_screen())

        except FileNotFoundError as e:
            print(f"Error: {e}")
            logo_label = tk.Label(
                header_frame,
                text="Logo Not Found",
                bg="#075E54",
                fg="white",
                font=("Segoe UI", 12, "italic")
            )
            logo_label.pack(side=tk.LEFT, padx=10)
        except Exception as e:
            print(f"Unexpected error loading logo: {e}")
        title_label = tk.Label(
```

4



```
class ChatClientGUI:
    def create_header(self):
        title_label = tk.Label(
            header_frame,
            text="Rizalian Chatroom",
            bg="#075E54",
            fg="white",
            font=("Segoe UI", 18, "bold"),
            )
        title_label.pack(side=tk.LEFT, padx=10)

    def open_menu_screen(self):
        menu_width = 300
        menu_frame = tk.Frame(self.window, bg="#FFFFFF", width=0, height=self.window.winfo_height())
        menu_frame.place(x=0, y=0, relheight=1, anchor="nw")

        for width in range(0, menu_width + 1, 10):
            self.window.update()
            menu_frame.config(width=width)
            time.sleep(0.01)

        back_button = tk.Button(
            menu_frame,
            text="Back",
            font=("Segoe UI", 12, "bold"),
            bg="#ECE5DD",
            fg="#000000",
            command=lambda: self.close_menu(menu_frame),
        )
        back_button.grid(row=0, column=0, padx=10, pady=10, sticky="W")

        menu_canvas = tk.Canvas(menu_frame, bg="#FFFFFF", highlightthickness=0)
```

Screenshots of program codes for Clients

5

```
class ChatClientGUI:
    def open_menu_screen(self):
            fg="#000000",
            command=lambda: self.close_menu(menu_frame),
        )
        back_button.grid(row=0, column=0, padx=10, pady=10, sticky="w")

        menu_canvas = tk.Canvas(menu_frame, bg="#FFFFFF", highlightthickness=0)
        menu_scrollbar = ttk.Scrollbar(menu_frame, orient="vertical", command=menu_canvas.yview)
        menu_scrollable_frame = tk.Frame(menu_canvas, bg="#FFFFFF")

        menu_canvas.create_window((0, 0), window=menu_scrollable_frame, anchor="nw")
        menu_canvas.configure(yscrollcommand=menu_scrollbar.set)

        menu_canvas.grid(row=1, column=0, sticky="nsew")
        menu_scrollbar.grid(row=1, column=1, sticky="ns")

        menu_frame.grid_rowconfigure(1, weight=1)
        menu_frame.grid_columnconfigure(0, weight=1)

        menu_options = ["Settings", "Chatbox Customization", "Font Customization"]
        for row, option in enumerate(menu_options):
            option_label = tk.Label(
                menu_scrollable_frame,
                text=option,
                bg="#ECE5DD",
                fg="#000000",
                font=("Segoe UI", 14),
                anchor="center",
                padx=10,
                pady=10,
                relief="solid",
```

6

```
class ChatClientGUI:
    def open_menu_screen(self):
                font=("Segoe UI", 14),
                anchor="center",
                padx=10,
                pady=10,
                relief="solid",
                borderwidth=1,
            )
            option_label.grid(row=row, column=0, padx=5, pady=5, sticky="ew")

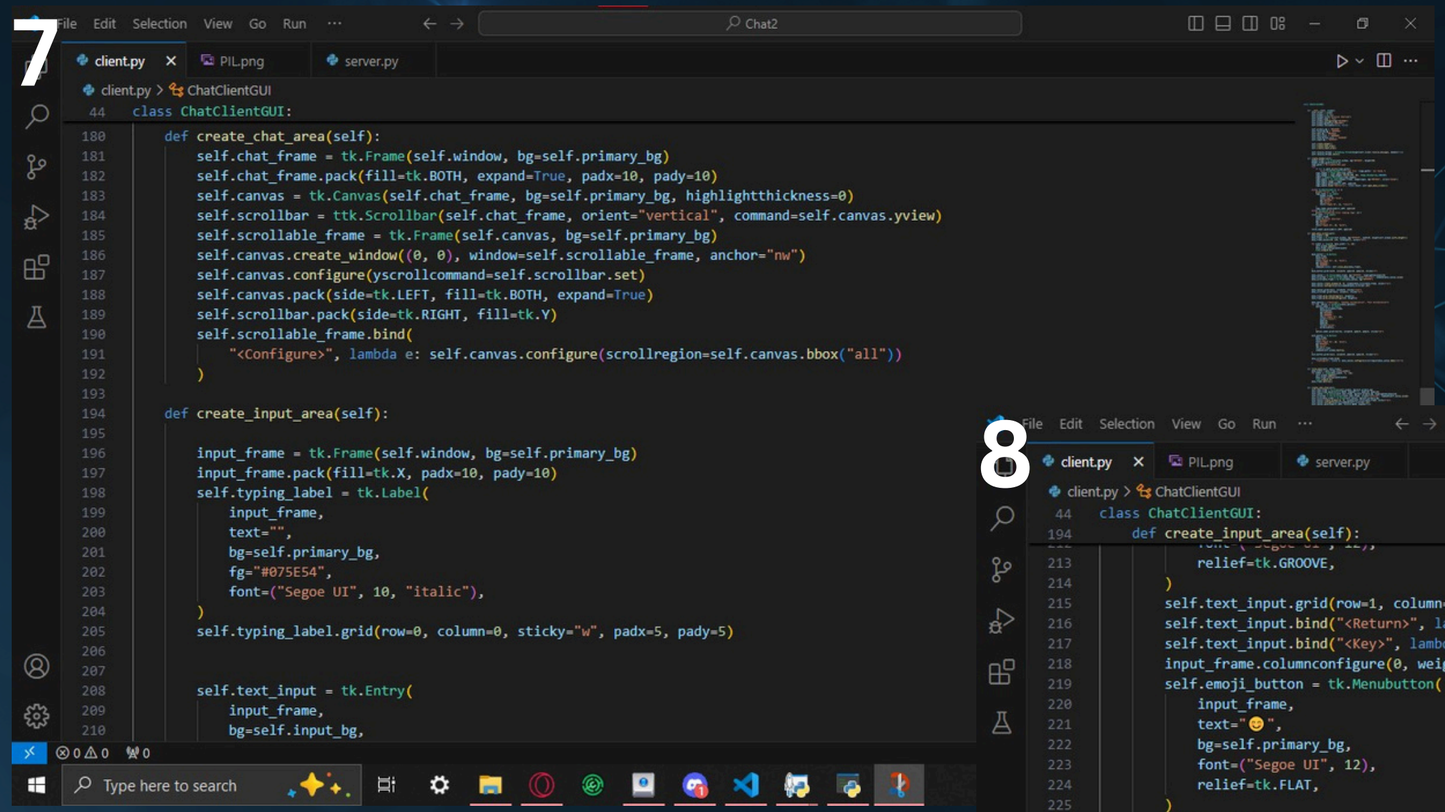
        exit_button = tk.Button(
            menu_frame,
            text="Exit",
            font=("Segoe UI", 10, "bold"),
            bg="#FF4C4C",
            fg="white",
            relief=tk.FLAT,
            command=self.window.destroy,
        )
        exit_button.grid(row=2, column=0, padx=10, pady=10, sticky="ew")

        menu_scrollable_frame.bind(
            "<Configure>", lambda e: menu_canvas.configure(scrollregion=menu_canvas.bbox("all"))
        )


    def close_menu(self, menu_frame):
        menu_width = menu_frame.winfo_width()
        for width in range(menu_width, -1, -10):
            self.window.update()
            menu_frame.config(width=width)
            time.sleep(0.01)
```

Screenshots of program codes for Clients

7



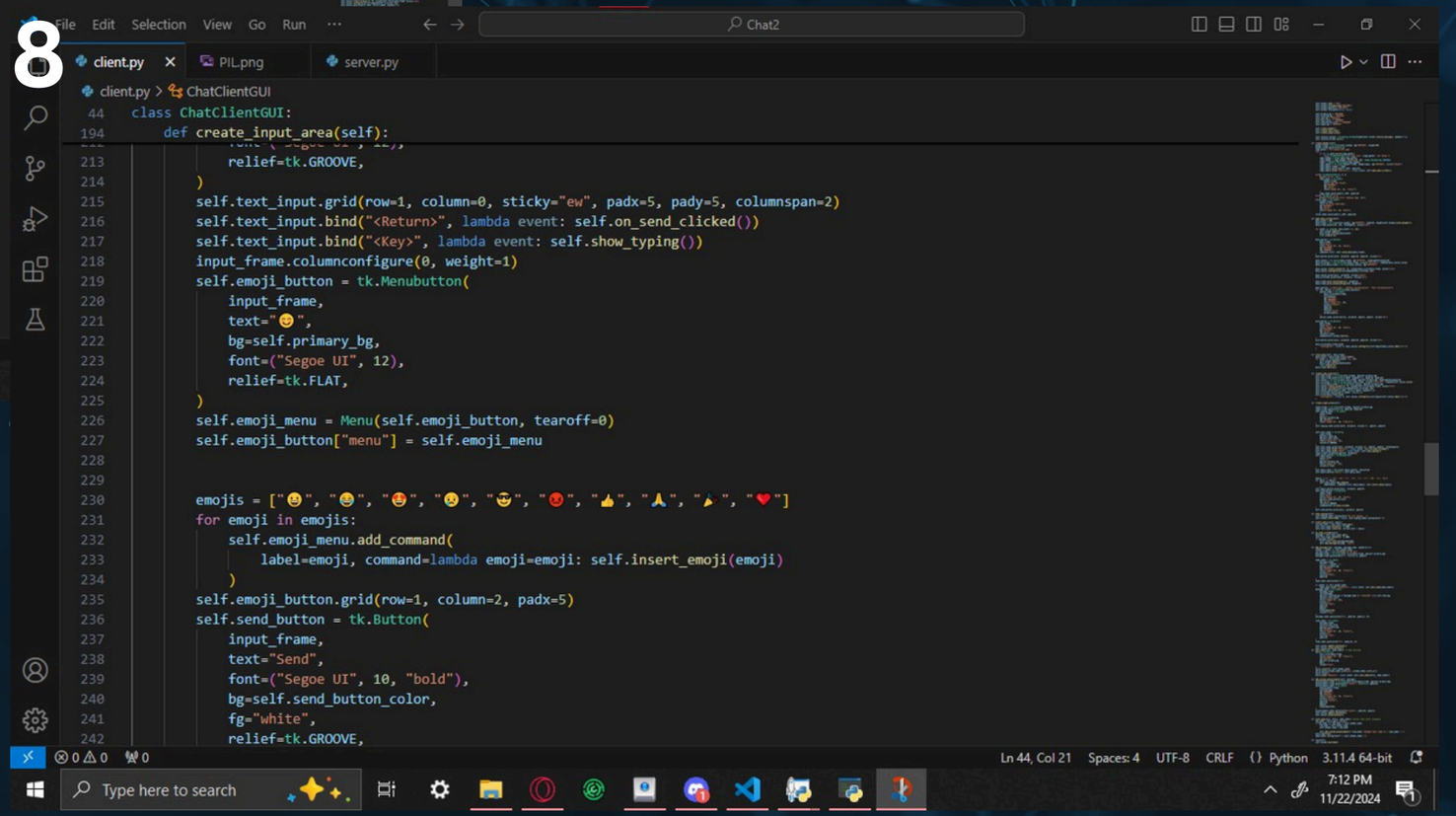
```
class ChatClientGUI:
    def create_chat_area(self):
        self.chat_frame = tk.Frame(self.window, bg=self.primary_bg)
        self.chat_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)
        self.canvas = tk.Canvas(self.chat_frame, bg=self.primary_bg, highlightthickness=0)
        self.scrollbar = ttk.Scrollbar(self.chat_frame, orient="vertical", command=self.canvas.yview)
        self.scrollable_frame = tk.Frame(self.canvas, bg=self.primary_bg)
        self.canvas.create_window((0, 0), window=self.scrollable_frame, anchor="nw")
        self.canvas.configure(yscrollcommand=self.scrollbar.set)
        self.canvas.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
        self.scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
        self.scrollable_frame.bind(
            "<Configure>", lambda e: self.canvas.configure(scrollregion=self.canvas.bbox("all"))
        )

    def create_input_area(self):

        input_frame = tk.Frame(self.window, bg=self.primary_bg)
        input_frame.pack(fill=tk.X, padx=10, pady=10)
        self.typing_label = tk.Label(
            input_frame,
            text="",
            bg=self.primary_bg,
            fg="#075E54",
            font=("Segoe UI", 10, "italic"),
        )
        self.typing_label.grid(row=0, column=0, sticky="w", padx=5, pady=5)


        self.text_input = tk.Entry(
            input_frame,
            bg=self.input_bg,
```

8

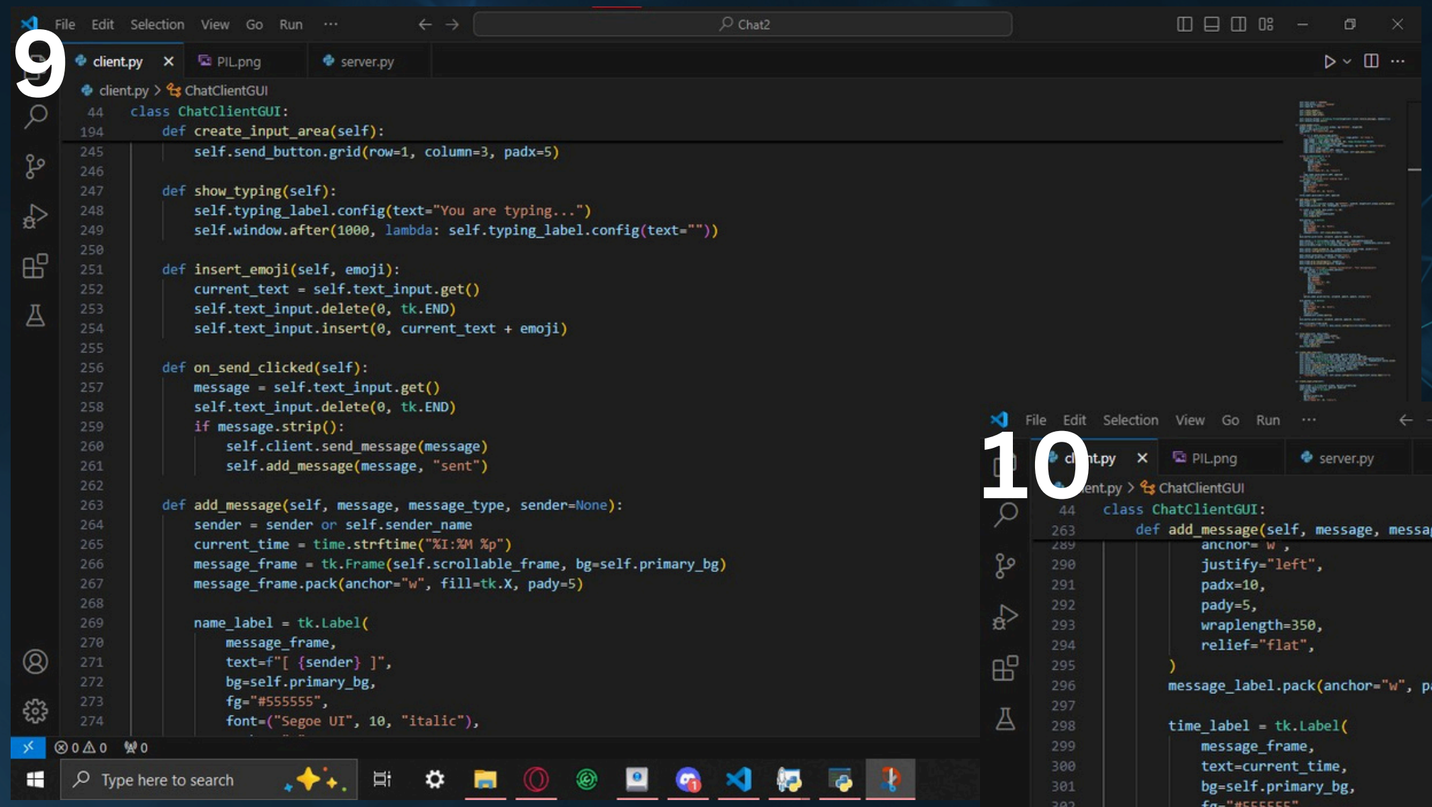


```
class ChatClientGUI:
    def create_input_area(self):
            relief=tk.GROOVE,
        )
        self.text_input.grid(row=1, column=0, sticky="ew", padx=5, pady=5, columnspan=2)
        self.text_input.bind("<Return>", lambda event: self.on_send_clicked())
        self.text_input.bind("<Key>", lambda event: self.show_typing())
        input_frame.columnconfigure(0, weight=1)
        self.emoji_button = tk.Menubutton(
            input_frame,
            text="😊",
            bg=self.primary_bg,
            font=("Segoe UI", 12),
            relief=tk.FLAT,
        )
        self.emoji_menu = Menu(self.emoji_button, tearoff=0)
        self.emoji_button["menu"] = self.emoji_menu


        emojis = ["😀", "😂", "😍", "😢", "😎", "😡", "👍", "🙏", "🎉", "❤️"]
        for emoji in emojis:
            self.emoji_menu.add_command(
                label=emoji, command=lambda emoji=emoji: self.insert_emoji(emoji)
            )
        self.emoji_button.grid(row=1, column=2, padx=5)
        self.send_button = tk.Button(
            input_frame,
            text="Send",
            font=("Segoe UI", 10, "bold"),
            bg=self.send_button_color,
            fg="white",
            relief=tk.GROOVE,
```

Screenshots of program codes for Clients

9



```
class ChatClientGUI:
    def create_input_area(self):
        self.send_button.grid(row=1, column=3, padx=5)

    def show_typing(self):
        self.typing_label.config(text="You are typing...")
        self.window.after(1000, lambda: self.typing_label.config(text=""))

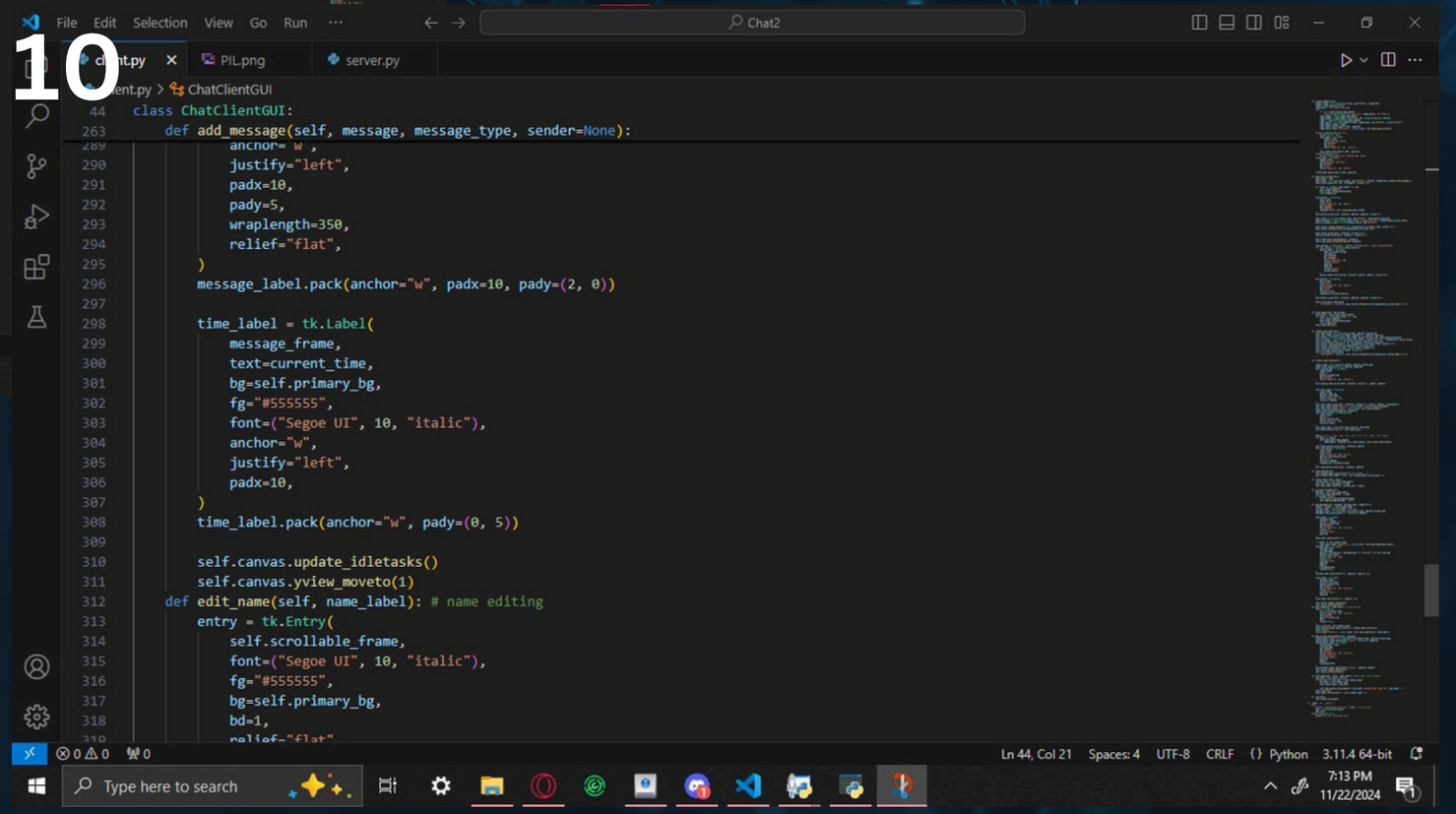
    def insert_emoji(self, emoji):
        current_text = self.text_input.get()
        self.text_input.delete(0, tk.END)
        self.text_input.insert(0, current_text + emoji)

    def on_send_clicked(self):
        message = self.text_input.get()
        self.text_input.delete(0, tk.END)
        if message.strip():
            self.client.send_message(message)
            self.add_message(message, "sent")

    def add_message(self, message, message_type, sender=None):
        sender = sender or self.sender_name
        current_time = time.strftime("%I:%M %p")
        message_frame = tk.Frame(self.scrollable_frame, bg=self.primary_bg)
        message_frame.pack(anchor="w", fill=tk.X, pady=5)

        name_label = tk.Label(
            message_frame,
            text=f"[ {sender} ]",
            bg=self.primary_bg,
            fg="#555555",
            font=("Segoe UI", 10, "italic"),
```

10



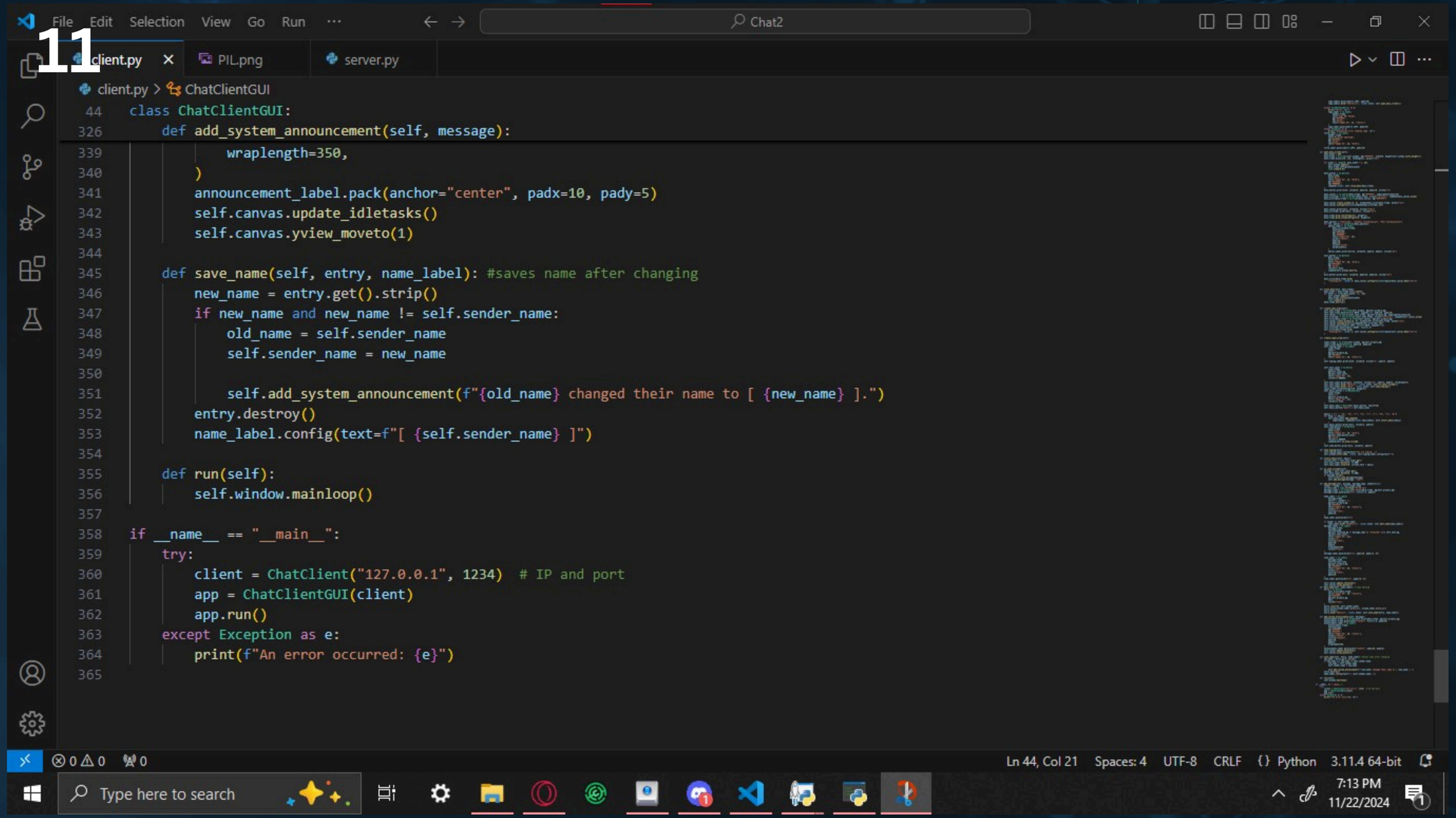
```
class ChatClientGUI:
    def add_message(self, message, message_type, sender=None):
            anchor="w",
            justify="left",
            padx=10,
            pady=5,
            wraplength=350,
            relief="flat",
        )
        message_label.pack(anchor="w", padx=10, pady=(2, 0))

        time_label = tk.Label(
            message_frame,
            text=current_time,
            bg=self.primary_bg,
            fg="#555555",
            font=("Segoe UI", 10, "italic"),
            anchor="w",
            justify="left",
            padx=10,
        )
        time_label.pack(anchor="w", pady=(0, 5))

        self.canvas.update_idletasks()
        self.canvas.yview_moveto(1)
    def edit_name(self, name_label): # name editing
        entry = tk.Entry(
            self.scrollable_frame,
            font=("Segoe UI", 10, "italic"),
            fg="#555555",
            bg=self.primary_bg,
            bd=1,
```

Screenshots of program codes for Clients

11



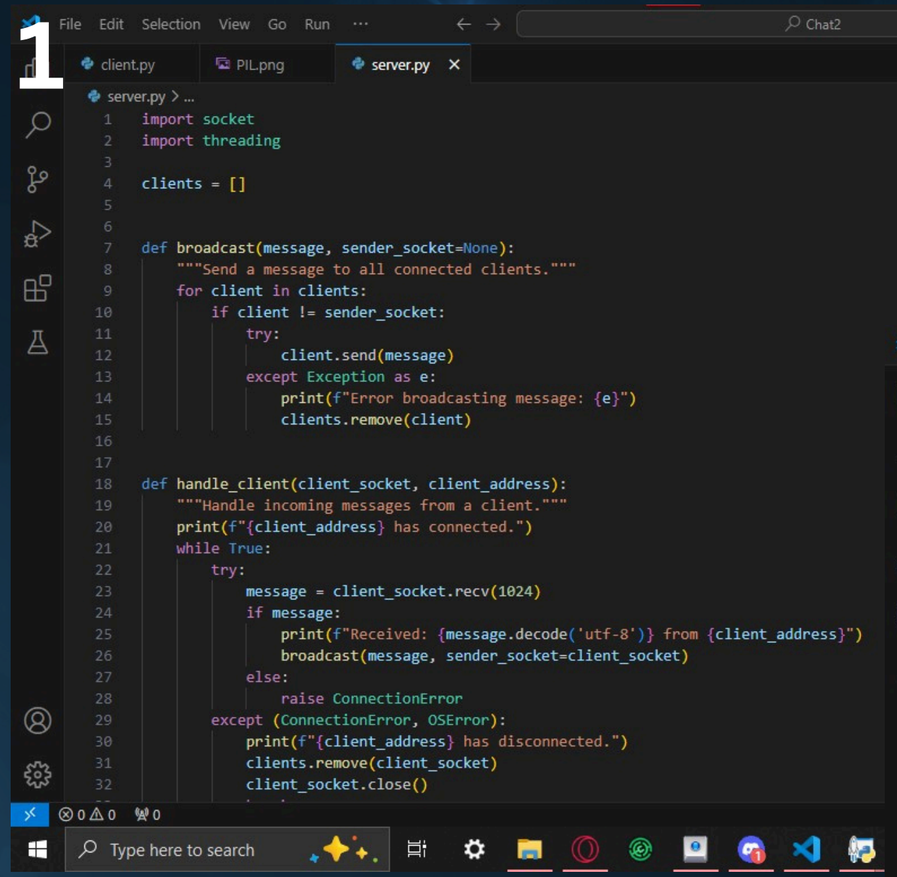
SERVER CODES



The provided code implements a simple chat server using Python's socket and threading libraries, allowing multiple clients to connect and communicate with each other. It listens for incoming messages from clients and broadcasts them to all other connected users, handling client disconnections gracefully.

Screenshots of program codes for the Server

1



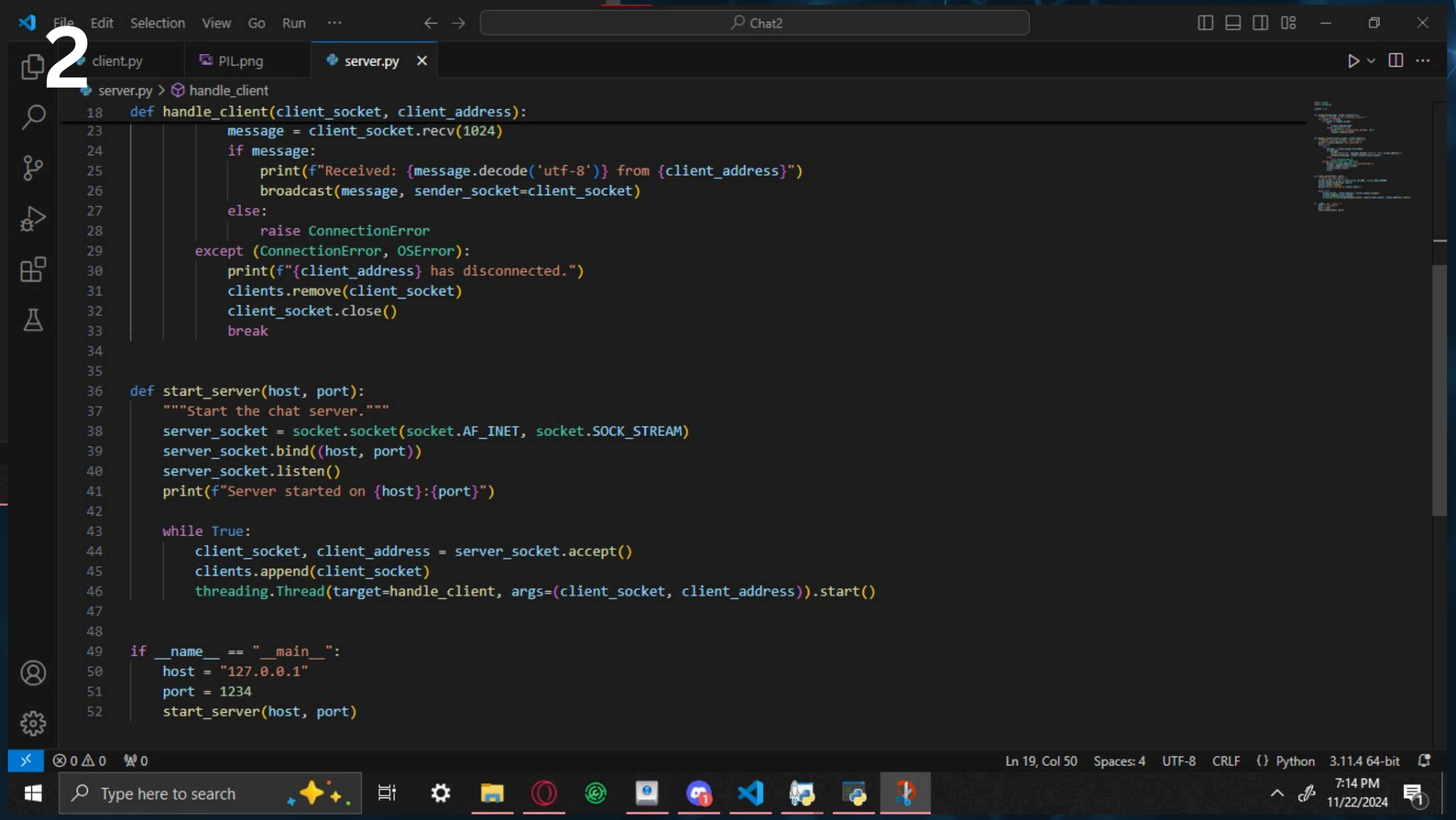
```python
import socket
import threading

clients = []


def broadcast(message, sender_socket=None):
    """Send a message to all connected clients."""
    for client in clients:
        if client != sender_socket:
            try:
                client.send(message)
            except Exception as e:
                print(f"Error broadcasting message: {e}")
                clients.remove(client)


def handle_client(client_socket, client_address):
    """Handle incoming messages from a client."""
    print(f"{client_address} has connected.")
    while True:
        try:
            message = client_socket.recv(1024)
            if message:
                print(f"Received: {message.decode('utf-8')} from {client_address}")
                broadcast(message, sender_socket=client_socket)
            else:
                raise ConnectionError
        except (ConnectionError, OSError):
            print(f"{client_address} has disconnected.")
            clients.remove(client_socket)
            client_socket.close()
```

2



```python
    def handle_client(client_socket, client_address):
                message = client_socket.recv(1024)
                if message:
                    print(f"Received: {message.decode('utf-8')} from {client_address}")
                    broadcast(message, sender_socket=client_socket)
                else:
                    raise ConnectionError
            except (ConnectionError, OSError):
                print(f"{client_address} has disconnected.")
                clients.remove(client_socket)
                client_socket.close()
                break


def start_server(host, port):
    """Start the chat server."""
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.bind((host, port))
    server_socket.listen()
    print(f"Server started on {host}:{port}")

    while True:
        client_socket, client_address = server_socket.accept()
        clients.append(client_socket)
        threading.Thread(target=handle_client, args=(client_socket, client_address)).start()


if __name__ == "__main__":
    host = "127.0.0.1"
    port = 1234
    start_server(host, port)
```

Thank You!